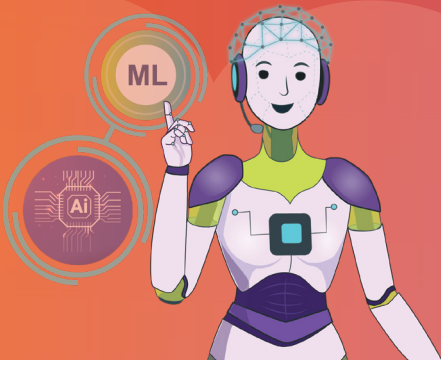




Introduction to Machine Learning Algorithms

ML is a powerful branch of artificial intelligence that enables systems to learn from data and improve their performance over time without explicit programming, revolutionising various industries through data-driven decision-making and prediction capabilities.



Have you ever used a smartphone to shop using an e-commerce app? What did you look for in the e-commerce app to purchase a product? Our product search constantly teaches the e-commerce application. It saves our preferences for future use based on the search product we have provided.

The e-commerce application cannot store all the data we enter due to the hundreds of search results it provides.

It uses a new programming method that is based on 'training' computers to learn. The computer learns based on inputs given by the user. Since it continuously improves and learns with each iteration, this programming approach is called **Machine Learning**. It is also considered a subset of **Artificial Intelligence or AI**.

Machine learning

Imagine having a reliable friend to whom you can always turn for help or information. Like anyone else, they too require time to learn and understand the subject matter. It is important to keep in mind that even this friend might not have all the answers at their fingertips. Instead, they learn from their experiences and progressively improve over time. Machine learning is similar, it is a computer that learns and improves on its own!

Back in 1959, an intelligent person named Arthur Samuel came up with the term 'Machine Learning'. He defined it as "A computer's ability to learn without being explicitly programmed". He wondered, what if we could make computers learn on their own without us having to instruct them exactly what to do?

Here is how it works: We give the computer some data, such as numbers, pictures, or words, and tell it what we want it to do with that data. The computer uses special algorithms (like little sets of instructions) to analyse the data and figure out how to solve the problem. However, here is the cool part: the more data the computer gets, the better it becomes at solving the problem.

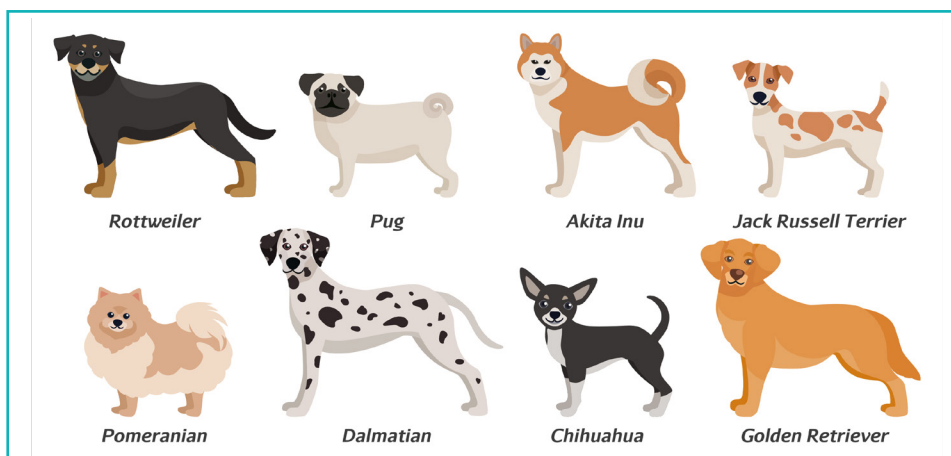
Imagine you are learning how to ride a bike. The first few times, it might be tricky, and you might fall a lot. However, as you keep practising, you get better and better until you can ride smoothly without falling. Machine learning is a bit like that - these algorithms are constantly fed with new and updated data, so that it can further learn from it and improve their operations, becoming increasingly intelligent over time.

These smart algorithms are used in a variety of everyday situations, such as when your parents ask their phone for directions or when a music app suggests new songs, you might like. It is like having a very helpful friend inside the computer who learns from the information it gets and applies that knowledge to make our lives easier and more interesting!

How Machine Learning Works?

'Machine Learning' is all about teaching machines or computers to learn and then make decisions, based on the data provided to it, similar to how we learn from our mistakes and experiences. This process of teaching machines can be broken down into the following steps:

For understanding purposes, let us assume that we want to teach a robot to recognise different breeds of dogs.



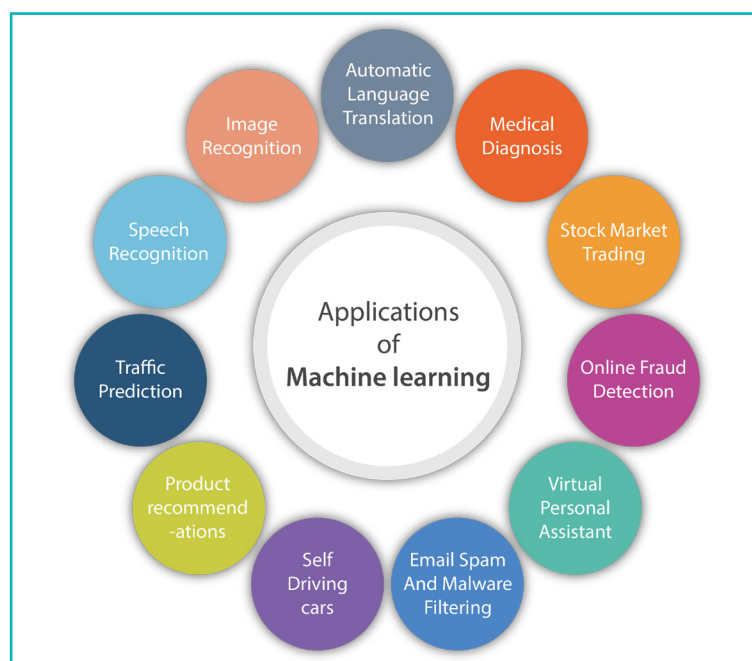
- a. We start by providing the robot with several pictures of different breeds of dogs, along with their breed names.
- b. The robot analyses the provided dog pictures along with their breed names. It will try to find patterns or features that distinguish one breed from another and learn from them.
- c. Once we provide enough input data (dog pictures), we can test out the robot by providing a random dog image and asking it to recognise its breed.
- d. Next, we provide feedback to the robot on whether it was able to correctly identify the breed or not. This feedback now acts as an input for the robot, helping it in improving its understanding of the topic.

This process is repeated multiple times, helping it learn more and improve its ability to identify dog breeds. Remember that the more data and examples we provide as input, the more accurate the machine-learning model can become.

Applications of Machine Learning

We are using machine learning in our daily lives even without realising it, such as Google Maps, Google Assistant, Alexa, etc. Below are some of the most trending real-world applications of Machine Learning:

- a. Machine Learning is used by search engines to provide us with relevant results.
- b. It is used by anti-spam software to identify potential spam messages or emails and filter them accordingly.
- c. It is also used to automatically detect online fraud transactions, keeping us protected from malicious activities.
- d. Used by biometric sensors and facial recognition systems for enhanced security.



Machine Learning Algorithms

An **algorithm** can be defined as a sequence of well-defined instructions for solving a particular problem or executing a specific task. Algorithms typically define a clear and systematic approach towards problem-solving in various domains.

Machine Learning Algorithms are algorithms that teach computers or machines to learn and make predictions or take decisions without being explicitly programmed. In machine learning, algorithms lay down their basic foundation. Programmed algorithms are used to receive and analyse input data as well as predict output values. As new data is fed into these algorithms, they learn and optimise their operations to improve performance, developing 'intelligence' over time.

ML algorithms help to solve different business problems like Regression, Classification, Forecasting, Clustering, Associations, etc.

Types of Machine Learning Algorithms

Machine Learning Algorithms are broadly classified into 4 types:

1. Supervised Learning
2. Semi-supervised Learning
3. Unsupervised Learning
4. Reinforcement Learning

Now, let us understand the different types of algorithms.

Supervised Learning

In **Supervised Learning**, we feed the algorithm with known datasets that include the inputs and the desired outputs. The goal here is to train the algorithm so that it can find a method to determine how to arrive at those inputs and outputs. Here, the algorithm learns by observing the input-output pairs and identifying patterns and relationships in the data.

For example, let us imagine we want to teach a machine how to identify whether an animal is a cat or a dog, and we start by providing it with pictures of dogs and cats and informing which is which. The machine analyses the input and the output and looks for patterns, such as the shape and size of cats and dogs, colour, height, etc. to distinguish between the pictures and improve its learning capabilities. The machine is then evaluated by providing it with sample pictures and checking whether it can guess the correct animal or not. This process continues with more and more datasets, improving the machine's accuracy over time.

Supervised learning is classified into two categories of algorithms:

- a. Classification:** A **Classification** problem is when the output variable is a category, such as 'red' or 'blue' or 'disease' and 'no disease'. Here, a machine learning program concludes from observed values and attempts to identify which category a new observation belongs to based on them.
- b. Regression:** A **Regression** problem occurs when the output variable is a real value, such as 'dollars' or 'weight'. It is used to handle regression problems where there is a linear relationship between input and output values.

Some common applications of Supervised Learning are – Image Classification, Fraud Detection, Spam Filtration, Speech Recognition, etc.

Algorithms Based on Supervised Learning:

- Regression
- Logistic Regression
- Naive Bayes Classifiers
- K-NN (k nearest neighbors)
- Decision Trees

Unsupervised Learning

In **Unsupervised Learning**, the algorithm learns to study data and identify patterns with no labelled data or human supervision. The machine is expected to explore the structure of the information, discover meaningful insights, detect patterns, interpret the given data, and address it on its own.

Going with the above example, we will now provide the algorithm with unlabelled images of cats and dogs. These images are entirely unknown to the model, and the machine is responsible for determining the patterns and categories of the object.

Unsupervised learning is classified into two categories of algorithms:

- a. Clustering:** A **Clustering** problem is where you want to discover the inherent groupings in the data, such as grouping customers based on their purchasing habits.
- b. Association:** An **Association** rule learning problem is where you want to discover rules that describe large portions of your data, such as persons who buy X also buy Y.

Some common applications of Unsupervised Learning are – Network Analysis, Recommendation System, Anomaly Detection, etc.

Algorithms based on Unsupervised Learning:

- K-means clustering
- KNN (k-nearest neighbors)
- Hierarchical clustering
- Anomaly detection
- Neural Networks

Semi-Supervised Learning

It is similar to supervised learning in that it learns from both labelled and unlabeled data for learning. In this method, the machine learns to identify objects using their associated labels and then applies that knowledge to identify unlabelled data.

Going with the above example, in Semi-Supervised Learning, we provide labelled pictures of cats and dogs as well as unlabelled. As the machine explores more pictures, it improves better at recognizing and classifying animals, even without explicit labels. They gain knowledge from both labelled and unlabelled examples.

Some common applications of Semi-supervised Learning are – Active Learning, Image and Video Recognition, Natural Language Processing (NLP), etc.

Reinforcement Learning

Reinforcement Learning works on a feedback-based process, in which an Artificially Intelligent Agent (A software component) automatically explore its surrounding by interacting with the environment, observing the current state, taking action, learning from experiences, and improving its performance. Unlike Supervised Learning, there is no labelled data here. Based on their actions, agents are also rewarded or penalized, giving them an understanding of the right and wrong courses of action.

Reinforcement Learning has applications in various domains, including Robotics, Game Theory, Autonomous Vehicles, Recommendation Systems, and more.

Algorithms based on Reinforcement Learning:

- Q-Learning
- Temporal Difference
- Monte-Carlo Tree Search

Exercise

 Tick mark ☒ the correct answer

1 Which Machine Learning Algorithm category does 'Classification' belong to?

- | | | | |
|--------------------|--------------------------|------------------|--------------------------|
| a. Supervised | ☐ | b. Unsupervised | ☐ |
| c. Semi-supervised | ☐ | d. Reinforcement | ☐ |

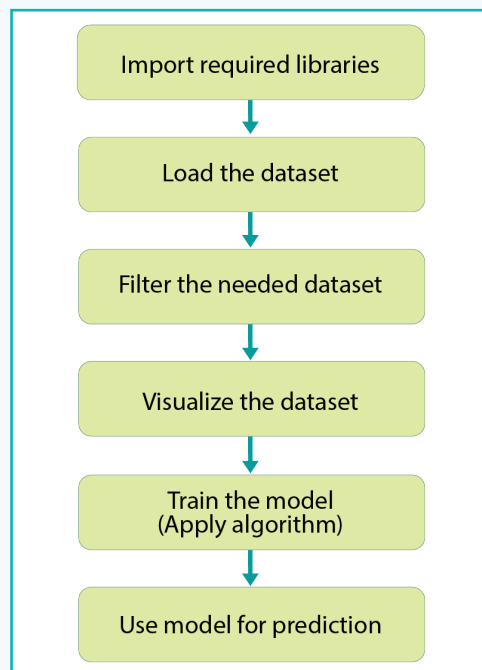
2 Which Machine Learning Algorithm category does 'Clustering' belong to?

- | | | | |
|--------------------|--------------------------|------------------|--------------------------|
| a. Supervised | ☐ | b. Unsupervised | ☐ |
| c. Semi-supervised | ☐ | d. Reinforcement | ☐ |

Activity

We are going to build a machine-learning program using Python. This program will try to predict the number of calories burned by an individual while walking for a specific period.

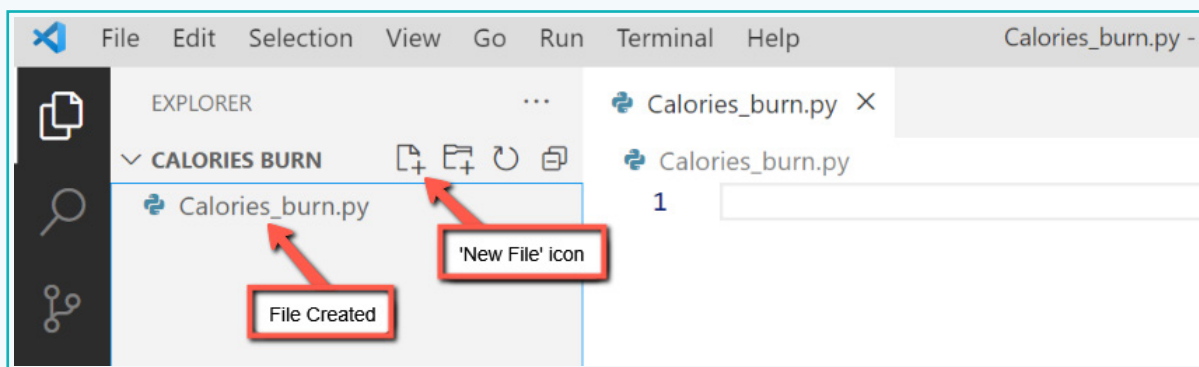
The entire project workflow can be summarised as follows:



1 Project Setup

We are going to use the VS Code to create our program.

- Create a folder “Calorie Burn” and open it in VS Code. Within this folder, create a Python file, ‘Calories_burn.py’ by clicking on the ‘New File’ icon available on the sidebar.



Activity

2 Installing Python Libraries/Modules

For our program to work, we will require a few libraries. In Python, these libraries are not available by default and must be installed manually using a Terminal. Fortunately, the VS Code provides an inbuilt terminal that we will use for installation purposes.

We are going to require 3 Python modules or libraries- **pandas**, **matplotlib**, and **scikit-learn** or simply **scikit**.

- a. **Pandas** is a powerful and widely used Python library for data manipulation and analysis. In machine learning, this library plays an important role in handling structured data by providing a tabular, spreadsheet-like data structure, called 'DataFrame'.
- b. **Matplotlib** is another widely used Python library for creating visualizations and plots. In machine learning, this library plays a crucial role for visualising and analysing data as well as understanding patterns within the data.
- c. **Scikit-learn** is a versatile Python library for implementing various machine-learning algorithms. It provides various tools for doing machine learning tasks such as classification, regression, clustering, model evaluation, etc.

Now, let us install the libraries using **pip**, which is a package installation and management tool in Python.

- d. In the terminal, execute the command 'pip install pandas' to install the pandas library.

```
PS D:\Calories Burn> pip install pandas
Collecting pandas
  Using cached pandas-2.0.3-cp311-cp311-win_amd64.whl (10.6 MB)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\
thon\python311\lib\site-packages (from pandas) (2.8.2)
```

- e. Similarly, install the other two libraries – 'matplotlib' and 'scikit-learn'

```
PS D:\Calories Burn> pip install matplotlib
Collecting matplotlib
  Using cached matplotlib-3.7.2-cp311-cp311-win_amd64.whl
Requirement already satisfied: contourpy>=1.0.1 in c:\user
thon\python311\lib\site-packages (from matplotlib) (1.1.0)
```

```
PS D:\Calories Burn> pip install scikit-learn
Collecting scikit-learn
  Using cached scikit_learn-1.3.0-cp311-cp311-win_amd64.
```

Now that our library installation is complete, next we will create a dataset.

Exercise

■ Fill in the blanks.

- 3 The 'DataFrame' belongs to the _____ library.
- 4 The _____ library is used to create visualizations and plots.

Activity

3 Preparing the DataSet

To create our dataset, we will use a '.csv' file. A .csv, also known as **Comma Separated Values** file, is a simple text file that is commonly used to store data in a tabular format (rows and columns), like an Excel spreadsheet.

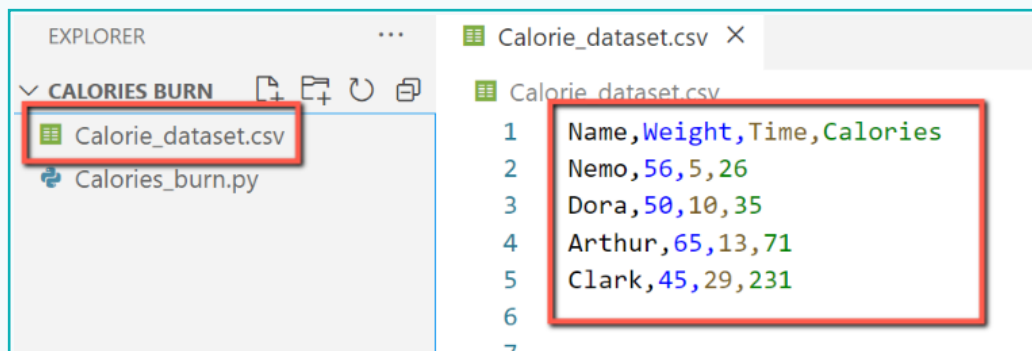
In a .csv file, each row represents a set of data, with each value separated by a comma (hence the name Comma Separated Values). The following represents a dummy .csv file for demonstration purposes.

Name	Age	Grade
Nemo	14	9
Dora	15	10
Barbie	13	8

For our program, we will create a .csv file and fill it up with some data. This file will act as our dataset.

Activity

- Create a file 'Calorie_dataset.csv' and in the file, define four columns: 'Name', 'Weight', 'Time', and 'Calories', as well as add some rows of data.



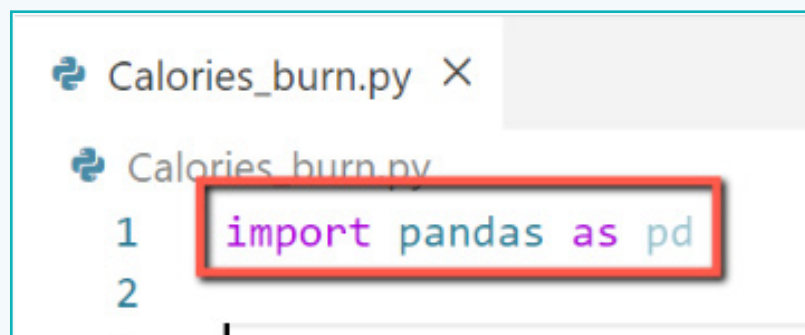
```
1 Name,Weight,Time,Calories
2 Nemo,56,5,26
3 Dora,50,10,35
4 Arthur,65,13,71
5 Clark,45,29,231
6
7
```

In the 'Calorie_dataset.csv' file, 'Weight' is displayed in 'Kg', 'Time' is displayed in 'minutes', and 'Calories' displays the 'number of calories burned' during this duration. For the dataset, you can also consider other factors, like height, age, body temperature, etc.

4 Loading the Dataset

To load the dataset into our Python program, we will require the 'Pandas' module. So, let us import the module first.

- a. Open the 'Calories_burn.py' file and, using the 'import' keyword, load the 'Pandas' library. We will also specify 'pd' as an alias for the 'Pandas' library.



```
1 import pandas as pd
2
```

Activity

To read from the .csv file, we will use the 'read_csv()' function. This function reads the data from the specified .csv file and creates a DataFrame. A **Pandas DataFrame** is a two-dimensional data structure that organises data in rows and columns, similar to a spreadsheet. The following figure shows a sample Pandas DataFrame.

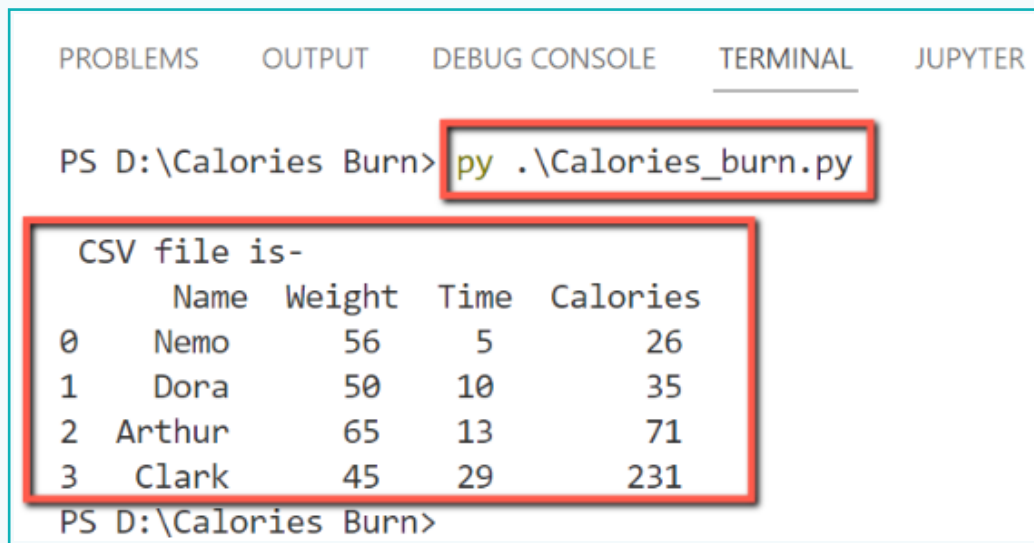
Pandas DataFrame			
Index	Columns		
	Value 1	Value 2	Value 3
	Value 4	Value 5	Value 6
	Value 7	Value 8	Value 9

- b. Create a variable 'df' and store the values from the .csv file using the 'read_csv()' function. We also display the 'df' variable, using the 'print()' method to ensure it contains the data read from the 'Calorie_dataset.' file.

```
Calories_burn.py •
Calories_burn.py > ...
1  import pandas as pd
2
3  df=pd.read_csv("Calorie_dataset.csv")
4  print("CSV file is-",'\n',df)
5
```

Activity

- c. Now, we will run our program to check the output. So, in the Terminal, execute your program using the command 'py Calories_burn.py'.



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL JUPYTER

PS D:\Calories Burn> py .\Calories_burn.py

CSV file is-
      Name  Weight  Time  Calories
0    Nemo     56     5      26
1    Dora     50    10      35
2  Arthur     65    13      71
3   Clark     45    29     231
PS D:\Calories Burn>
  
```

5 Indexing and Selecting data with Pandas

Indexing in Pandas

Indexing in Pandas means simply selecting particular rows and columns of data from a DataFrame. We use the 'Indexing operator' represented by square brackets ' [] ' to index and select data in pandas.

Considering 'df' as a DataFrame,

- df['column_name'] will select the specified column name.
- df[['column_1', 'column_2', 'column_3']] will select multiple column names.

Now, let us filter the data in the .csv file and select only 2 columns, 'Time' and 'Calories' and save it in two separate variables. Later, we will train our model on these selected columns.

Activity

- c. In the Python file, select the two columns using the 'indexing operator' and store them on separate variables, 'time' and 'cal'. Print the values as well using the 'print()' method.

```
Calorie_dataset.csv  Calories_burn.py X
Calories_burn.py > ...
4  print('\n',"CSV file is-",'\n',df)
5
6  time=df['Time']
7  cal=df['Calories']
8
9  print('\n','Time Data-'\n',time)
10 print('\n','Calories Data-'\n',cal)
11
```

- d. Run your program to check the output.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  JUPYTER

Time Data-
0      5
1     10
2     13
3     29
Name: Time, dtype: int64

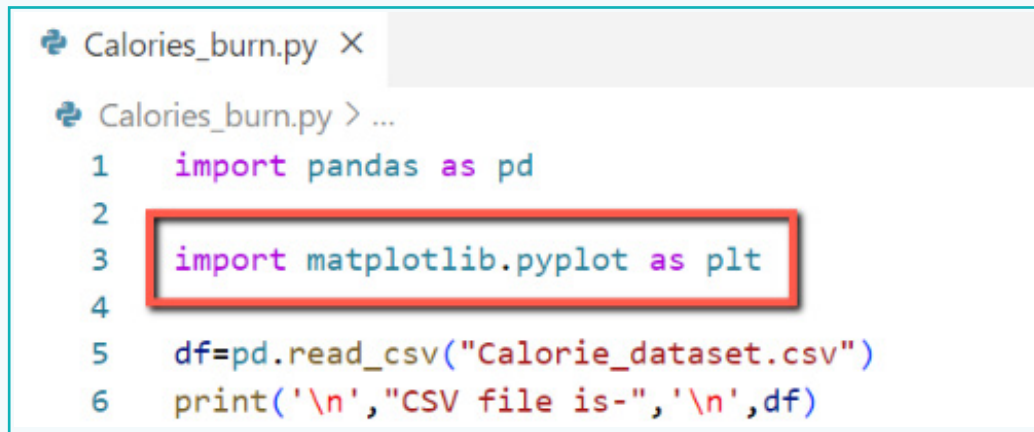
Calories Data-
0     26
1     35
2     71
3    231
Name: Calories, dtype: int64
PS D:\Calories Burn>
```

Activity

6 Visualizing the Data

To visualize the data, we will require the 'matplotlib' Python library. We had already installed the library at the beginning of the activity, so we can just import it into the Python file.

a. Import the 'matplotlib' library at the top of the program.



```
Calories_burn.py X
Calories_burn.py > ...
1  import pandas as pd
2
3  import matplotlib.pyplot as plt
4
5  df=pd.read_csv("Calorie_dataset.csv")
6  print('\n', "CSV file is-", '\n', df)
```

Here, we are importing the 'matplotlib.pyplot' module and giving it an alias 'plt'. The 'pyplot' module is a component of the 'matplotlib' library and is used to create various types of plots and visualizations like line plots, scatter plots, bar charts, etc.

Now that the 'matplotlib' library is imported, we can start creating the visualization. However, let us get familiar with a few matplotlib methods.

- **title():** It is used to set the title of the plot.
- **xlabel():** It is used to set a label for the x-axis.
- **ylabel():** It is used to set a label for the y-axis.
- **plot():** It is used to create a simple line plot or a line chart.
- **scatter():** A scatter plot is a diagram where each value in the data set is represented by a dot. This method is used to draw a scatter plot.
- **grid():** It is used to add grid lines to the plot.
- **show():** It is used to display the plot on the screen.
- First, we will create a simple line plot or a line chart using the 'plot()' method. This method takes two arguments that represent the data points for the x-axis and the y-axis, respectively.

Activity

- b. First, we will create a simple line plot or a line chart using the 'plot()' method. This method takes two arguments that represent the data points for the x-axis and the y-axis, respectively.

```
5 df=pd.read_csv("Calorie_dataset.csv")
6 print('\n',"CSV file is-",'\\n',df)
7
8 time=df['Time']
9 cal=df['Calories']
10
11 plt.plot(time, cal)
12
```

As can be seen, we are passing the 'time' and 'cal' variables as parameters, since we want to create the line plot with the 'time' and 'cal' values, representing the data points for the x-axis and the y-axis.

- c. Next, we will use the 'title()' method to provide a name for the line plot.

```
plt.plot(time, cal)
plt.title('Calories burned prediction',color='green',fontsize=16)
```

- d. Next, we will set the labels for the x-axis and y-axis using the 'xlabel()' and 'ylabel()' methods.

```
plt.title('Calories burned prediction',color='b',fontsize=16)
plt.xlabel('Time in minutes',color='red',fontsize=14)
plt.ylabel('Calories burned',color='red',fontsize=14)
```

- e. Now, we will use the 'scatter()' method to create a **scatter plot**. A scatter plot is a diagram where each value in the data set is represented by a dot.

```
15 plt.xlabel('Time in minutes',color='red',fontsize=14)
16 plt.ylabel('Calories burned',color='red',fontsize=14)
17
18 plt.scatter(time,cal,color = 'blue', linestyle = 'solid')
19
```


Activity

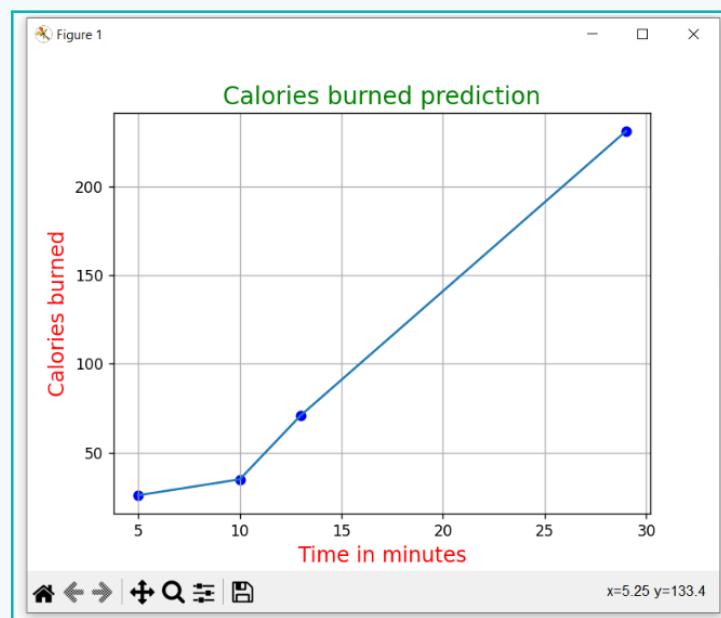
f. Add grid lines to the line plot using the 'grid()' method.

```
17  
18 plt.scatter(time,cal,color = 'blue',  
19  
20 plt.grid()  
21
```

g. Finally, display the line plot on the screen using the 'show()' method.

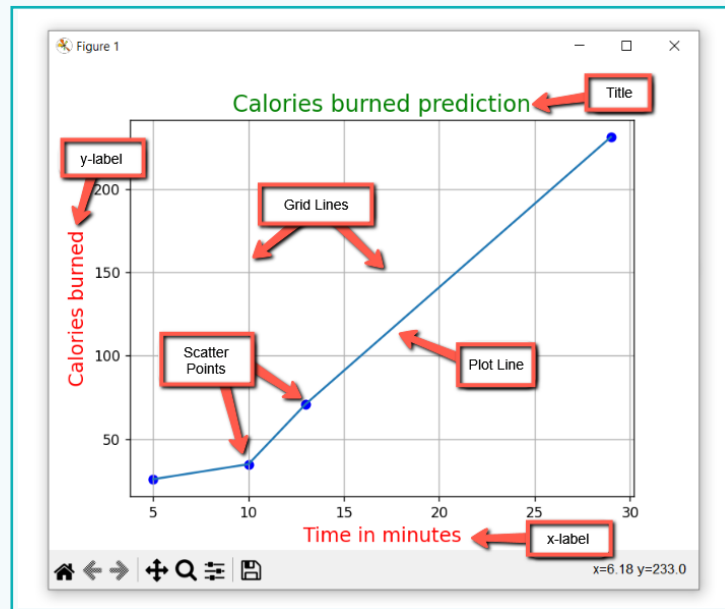
```
18 plt.scatter(time,cal,color = 'blue', :  
19  
20 plt.grid()  
21  
22 plt.show()  
23
```

h. Execute the program to check the output.



Activity

i. The following figure shows the output with the labels provided.



Exercise

Tick mark ☒ the correct answer

5 Which 'Pandas' function is used to read from a.csv file?

a. readCsv() ☐

b. read_csv() ☐

c. csvRead() ☐

d. csv_read() ☐

6 Which set of brackets represents the 'Indexing Operator'?

a. [] ☐

b. {} ☐

c. () ☐

d. <> ☐

Activity

7 Training the Model

To train the model, we are going to use the 'Linear Regression' algorithm, which is based on Supervised Learning. So, let us import the 'Linear Regression' class from the 'scikit-learn' library so that we can use it to create and train a 'Linear Regression' model.

The **Linear Regression** algorithm predicts the output based on one or more input features with the assumption that a linear relationship exists between the input and output target variable.

Consider the following chart:

Hours - 8	% Marks - 60
Hours - 10	% Marks - 70
Hours - 14	% Marks - 75
Hours - 12	% Marks - 72
Hours - 13	% Marks - ???

Can you predict the % Marks for next 13 hours? Yes, it will be either 73 or 74.

This algorithm works on the relationship between the continuous variables, which have 'an infinite number of values between any two values'. Here, 'Hours' and 'Marks' are the continuous variables.

Some popular applications of linear regression are analysing trends and sales, weather forecasting, salary prediction, and real estate prediction.

a. In the 'Calories_burn.py' file, import the 'Linear Regression' class.

```
Calories_burn.py X
Calories_burn.py > ...
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 from sklearn.linear_model import LinearRegression
4
5 df=pd.read_csv("Calorie_dataset.csv")
```

Activity

a. Now, we need to create a 'Linear Regression' model object using the 'LinearRegression()' method.

```
20 plt.grid()
21 plt.show()
22
23 model=LinearRegression()
24
```

Next, we need to train the 'Linear Regression' model object, with the given training data. This is done using the 'fit()' method, which is available as part of the 'scikit-learn' library.

The fit() method

This method belongs to the 'Scikit-learn' library and it is used to train the algorithm on the training data. In simple terms, the 'fit()' method uses the training data as an input to train the machine learning model, allowing it to learn patterns and relationship in the data so that it can make predictions on new data.

To use the 'fit()' method, we need to call it from an existing instance of a machine learning model, like Linear Regression, SVM, etc.

Syntax for the 'fit()' method:

model.fit(x_train, y_train)

- **Model:** It refers to the instance of the machine learning model.
- **x_train:** It refers to the feature data or input used for training the model. This parameter should be a 2-dimensional array.
- **y_train:** It refers to the target data or output used for training the model. This parameter should be a 1-dimensional array.

As per the syntax of the 'fit()' method, the 'x_train' parameter should be a 2-dimensional array. So, we need to convert the existing 'Time' column data, which serves as the input, from a 1-Dimensional array into a 2-Dimensional Numpy array. This can be done by using a combination of the 'values' property and the 'reshape()' method.

The 'values' property is used to access data from a Pandas DataFrame or Series, as 'Numpy' arrays.

The 'reshape()' method in NumPy is used to change the shape of a NumPy array without changing its data. We can convert a 1D array to a 2D or 3D array and vice versa if the total number of elements in the array remains the same.

Activity

a. Let us use the 'values' property and 'reshape()' method to do the conversion.

```
22
23     model=LinearRegression()
24
25     time=time.to_numpy()
26     cal=cal.to_numpy()
27
```

Here,

- **df['Time']:** Refers to the 'Time' column in the DataFrame.
- **df['Calories']:** Refers to the 'Calories' column in the DataFrame.
- **values:** Extracts the 'Time' column data from a pandas Series as a 1-Dimensional NumPy array.
- **reshape(-1, 1):** Converts the 1-Dimensional 'Time' array into a 2-Dimensional array with a single feature. This is an important step since the 'Linear Regression' model expects the input data in a 2-Dimensional format. Here, '-1' indicates the dimension is unknown and we want Numpy to calculate it for us and '1' indicates that the resulting 2-Dimensional array should have 1 column.

Now, let us use the 'fit()' method.

b. Using the 'Linear Regression' model object, we will invoke the 'fit()' method.

```
27
28     time=df['Time'].values.reshape(-1,1)
29     cal=df['Calories'].values
30
31     model.fit(time,cal)
32
```

8 Data Prediction

Predict () function

We have trained our model based on the data available in the .csv file, hence, the data in the CSV file is called the **Trained Data** in our project. The predict() function returns the predicted output for the input data provided to the trained model.

Python's 'predict()' function enables us to predict the data values based on the trained model, i.e., it makes predictions for new data points. The 'predict()' function accepts only a single argument, which is usually the data to be tested.

Activity

- a. Create a variable 'prediction_time' and store an integer on it. This integer represents the amount of time for which we want to predict the calorie burn.

```
31 model.fit(time,cal)
32
33 prediction_time = 20
34
```

- b. Now, we can use the 'predict()' method to make the forecast. This method, however, requires a 2-Dimensional object. So, we need to put the 'prediction_time' variable within double square brackets, which will turn the stored integer value into a 2-Dimensional array.

```
32
33 prediction_time = 20
34
35 predicted_calories = model.predict([[prediction_time]])
36
37
```

```
35 predicted_calories = model.predict([[prediction_time]])
36
37 print("Predicted Calories Burned for {} minutes: {:.3f} calories".format(prediction_time,predicted_calories[0]))
38
```

Here,

- **{ }**: Represents the placeholder where values will be displayed.
- **{:.3f}**: Specifies that the floating-point value to be formatted to three decimal places.
- **format()**: This method is used to insert values in the specified placeholders.

Exercise

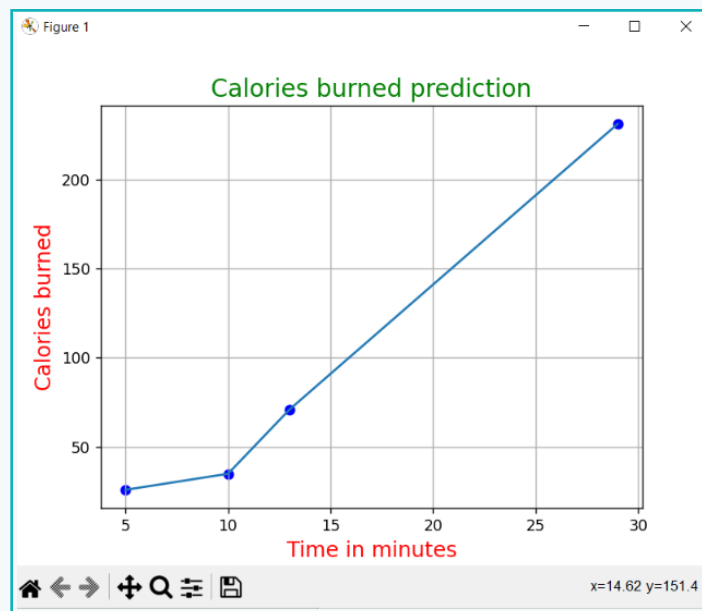
■ Fill in the blanks.

- 7 The _____ property is used to access data from a Pandas DataFrame.
- 8 The _____ method is used to change the shape of a Numpy Array.

Activity

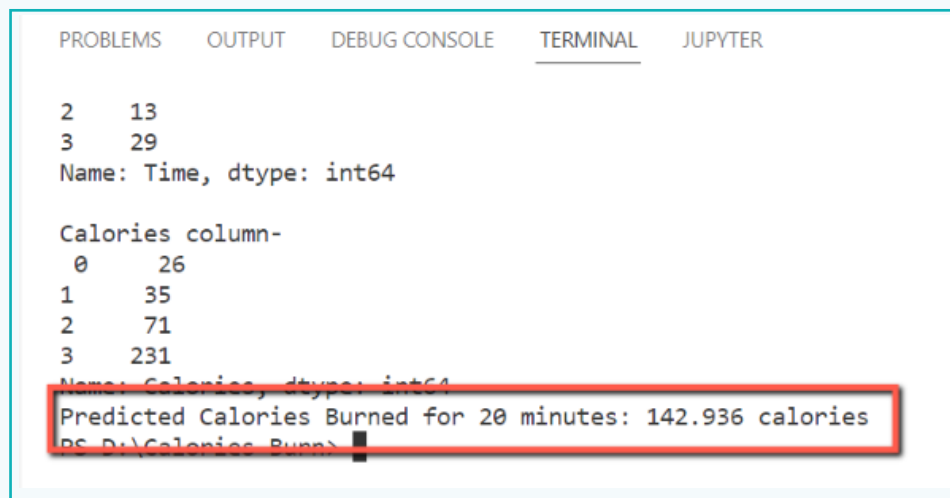
■ Checking the Output

- a. Execute the program to check the output. First, we will have the 'Line Plot' displayed.



Activity

- b. Now, if we close this graphical representation, we will be able to view the predicted result on the terminal.



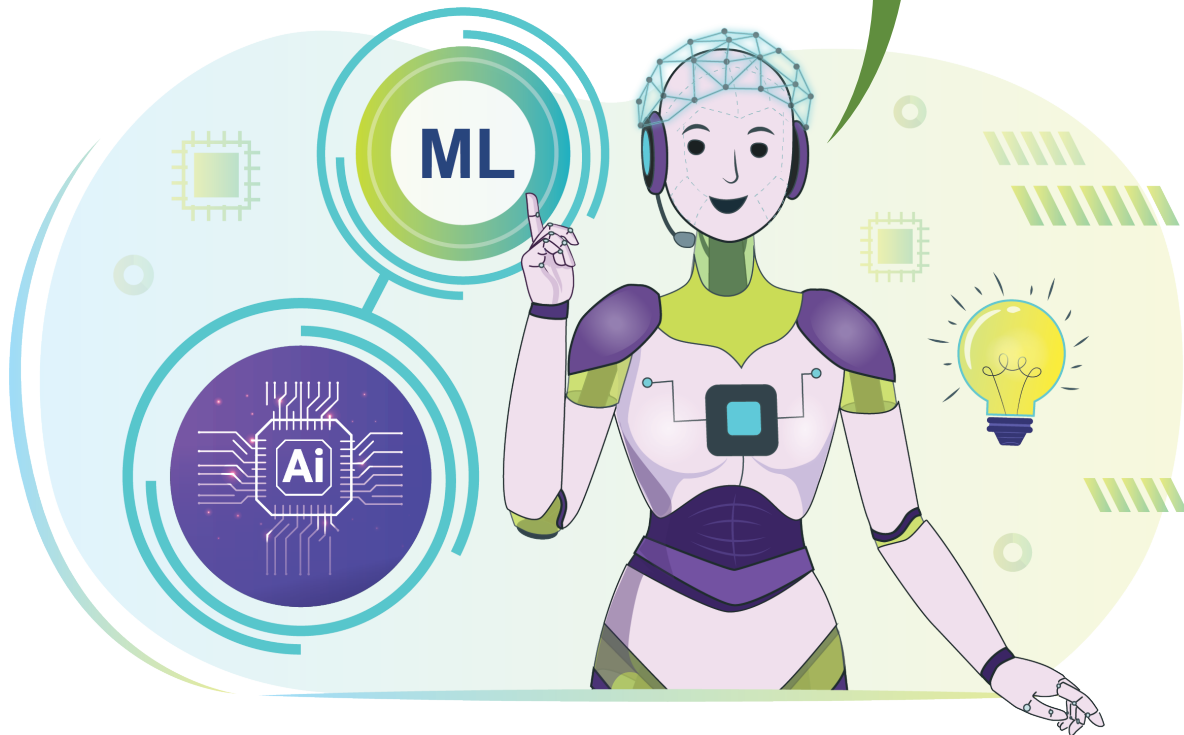
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL JUPYTER

2    13
3    29
Name: Time, dtype: int64

Calories column-
0     26
1     35
2     71
3    231
Name: Calories, dtype: int64
Predicted Calories Burned for 20 minutes: 142.936 calories
PS D:\Calories_Burn>
```

The lesson introduces machine learning algorithms and explains how they enable systems to learn from data without the need for explicit programming. It covers applications in various industries, including e-commerce, fraud detection, and biometric sensors. The process of machine learning is illustrated by teaching a robot to recognise dog breeds. The lesson categorizes machine learning algorithms into four types: 'Supervised Learning', 'Unsupervised Learning', 'Semi-Supervised Learning', and 'Reinforcement Learning' with real-world applications for each category. A practical activity guides learners through the creation of a machine learning program with Python using libraries such as pandas, matplotlib, and scikit-learn to prepare and visualise the dataset, train the model using linear regression, and make predictions. Overall, the lesson introduces machine learning and its practical implementation.

Fantastic job, everyone! The power of machine learning is right at your fingertips. Explore algorithms, tackle challenges, and keep pushing your skills to the next level. Embrace the possibilities of AI as you progress on your learning path!



Tech Flash

- **Artificial Intelligence** : It is a field where machines are trained to learn from their experiences, adapt to new inputs and perform activities that typically require human intelligence. It refers to giving machines the ability to simulate human intelligence with the help of machine learning algorithms.
- **Neural Network** : A subset of Artificial Intelligence developed to teach computers to process data in the same way that neural networks in human brains do. It is also referred to as 'Artificial or Simulated Neural Networks'.